REPOSITORY ANALYSIS

YouVersion Platform SDK for .NET

An in-depth engineering review of architecture, code quality, testing, CI/CD, and documentation for the `YouVersionPlatformDotNetSDK` repository.

Branch reviewed: Development

Latest commit at review time: 56b5100 — "One last update to the documentation."

Tagged releases: v0.1.0, v0.1.1, v0.1.2

Date: 2026-07-17

Prepared by: Claude Code (Sonnet 5), at the request of Kevin Forshey

1. Executive Summary

This is an unofficial, community-built .NET SDK for the YouVersion Platform REST API. It ships five NuGet packages layered from plain domain models up through OAuth/PKCE-authenticated HTTP clients, business-logic services, and Blazor UI components, backed by a matching working sample app (`PlatformTestApp`).

For a solo/small-team open-source SDK, the engineering discipline on display is well above what this category of project usually reaches. The standout traits are an **enforced** architectural boundary (a real test fails the build if it's violated, not just a diagram), a genuinely hard problem — OAuth/PKCE plus YouVersion's multi-shaped Data Exchange consent callback — solved carefully and documented with the scars from getting it wrong, and a changelog detailed enough to double as an incident log for a solo maintainer's future self.

11	124	5	0
PROJECTS (6 SHIPPING, 5 TEST)	C# SOURCE FILES	NUGET PACKAGES PUBLISHED	TODO / FIXME / HACK MARKERS

The gaps are mostly what you'd expect from a project that hasn't yet had external contributors or an incident: no dependency/security scanning, one test project that's quietly never run in CI, a couple of stale documentation claims, and complexity concentrated in a few large hand-rolled files (`PlatformTestApp/Program.cs`, `BookCatalog.cs`) that will get harder to change safely over time. None of it is structural — all fixable in isolated, low-risk changes.

2. Architecture

The solution is a clean, strictly-layered dependency chain:

```
Platform.API.Models        (POCO/records, zero dependencies)
        ^
        |
Platform.API             (typed HTTP clients, OAuth+PKCE, caching decorators, DI
wiring)
        ^
        |
Platform.SDK.Services        (service abstractions, BibleReaderStateService)
        ^
        |
Platform.SDK.Components     (Razor components: pickers, BibleReader, YouVersionAuth)
        ^
        |
PlatformTestApp              (Blazor Server sample host, OAuth endpoints)
```

A sixth package, `YouVersion.UsfmReferences` (an independent C# port of YouVersion's own `usfm-references` Python library), sits off to the side as a dependency of `Platform.API` and is versioned separately since it tracks the upstream project's own release cadence rather than the Platform.* packages' cadence.

> **STRENGTH — ARCHITECTURE IS ENFORCED, NOT JUST DOCUMENTED**
>
> `Platform.API.Tests/Architecture/ApiClientBoundaryTests.cs` greps `Platform.SDK.Components` and `PlatformTestApp` source for direct references to `Platform.API.Clients` types (`IBibleClient` , `IPassageClient` , `IHighlightClient`) and fails the build if any UI-layer file bypasses `Platform.SDK.Services` to talk to the HTTP client layer directly. Most READMEs draw this diagram; almost none of them enforce it in CI. This one does.

> **IMPROVEMENT — THE FITNESS TEST IS REGEX-OVER-SOURCE-TEXT, NOT A REAL DEPENDENCY CHECK**
>
> LOW
>
> The boundary test works by pattern-matching raw file text (`@"\bI(?:Bible|Passage|Highlight)Client\b"`), which is a reasonable zero-dependency approach but is easy to accidentally defeat (a type alias, a fully-qualified name split across lines, or a new client interface that doesn't match the three hardcoded names silently isn't caught). A short-to-medium-term upgrade would be to walk `ProjectReference` /compiled-assembly metadata (e.g. via `System.Reflection.Metadata` or Roslyn) instead of text patterns, or at minimum add a code

> comment next to `forbiddenPatterns` flagging it as the single place to update when a new client interface is added.

2.1 Package boundaries and "install the highest package" model

The README's package table doubles as a decision tree ("install the highest package that matches what you're building — its dependencies pull in everything below it"). This is a small thing done well: most multi-package SDKs make the consumer figure out the dependency graph themselves.

3. Code Quality

STRENGTH — QUALITY GATES ARE LOAD-BEARING, NOT ASPIRATIONAL

Every shipping project has `<Nullable>enable</Nullable>` and `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` set at the project level (not just suggested in a style guide). `Directory.Build.props` centralizes MinVer-based versioning and deterministic/SourceLink builds repo-wide instead of duplicating that boilerplate across five `.csproj` files. `GenerateDocumentationFile` is on, and public members are consistently XML-documented — `<NoWarn>CS1591</NoWarn>` only suppresses the "missing doc comment" warning rather than being used as a blanket escape hatch.

STRENGTH — THE OAUTH CLIENT SHOWS REAL PRODUCTION HARDENING

`OAuthBearerTokenHandler` single-flights concurrent token refreshes behind a shared `Task` and a lock, specifically to avoid the documented failure mode where `HttpClientFactory`'s cached handler chain lets many concurrent requests each independently race to refresh — and invalidate — the same refresh token. `ValidateState` uses `CryptographicOperations.FixedTimeEquals` for CSRF state comparison. PKCE verifier/challenge generation uses `RandomNumberGenerator` and SHA-256, not a weaker RNG. This is the kind of detail that's usually missing from "SDK wraps an API" projects and only shows up after a real incident — here it's present from the start.

STRENGTH — CACHING DECORATOR PATTERN IS CLEAN AND COMPOSABLE

`CachingBibleClient` / `CachingPassageClient` wrap the raw HTTP clients behind `HybridCache` (L1 in-memory + optional Redis L2), resolved by concrete type to avoid a circular DI dependency through the shared interface, and swapped in via `services.Replace(...)` only when `AddYouVersionCaching()` is explicitly opted into. The whole-index cache-once / slice-in-memory strategy for books/chapters/verses means a single cached fetch per Bible version serves every finer-grained lookup, which is a genuinely good design decision, not just "wrap it in a cache."

STRENGTH — EXCEPTION DESIGN DISTINGUISHES REAL FAILURE MODES

`YouVersionApiException` (non-2xx wire response) is intentionally kept separate from the newer `YouVersionEmptyResponseException` (2xx with an empty/unparsable body) — the changelog records this as a deliberate fix after the two were originally conflated by overloading `HttpStatusCode.OK` / `NotFound` to mean "empty," which is exactly the kind of exception-type conflation that makes API client error handling unreliable for consumers.

IMPROVEMENT — PROGRAM.CS CONCENTRATES TOO MUCH BRANCHING LOGIC INLINE

MEDIUM

`PlatformTestApp/Program.cs` is ~330 lines, and its single `app.Use(async (ctx, next) => ...)` middleware lambda handles five distinct callback shapes (code callback, standalone Data Exchange callback, direct/browser identity callback, the dev-only synthetic sign-in shortcut, and pass-through) via sequential `if` blocks matched on query-string keys. The in-line comments explaining *why* each branch exists are excellent, but the branching itself is exactly the shape that becomes an untestable God-middleware as more OAuth callback variants get added. Since this logic isn't test-covered today (it's sample-app code, not library code), extracting each branch into its own named method or minimal-API endpoint would make it unit-testable and reduce the risk of one branch's `return` being accidentally skipped when a new branch is inserted above it.

IMPROVEMENT — A FEW LARGE, SINGLE-PURPOSE FILES ARE DUE FOR A SPLIT

LOW

`YouVersion.UsfmReferences/BookCatalog.cs` (~12.3 KB) and `Reference.cs` (~12.6 KB) are large by the standards of the rest of the codebase (most files are under 2 KB). For `BookCatalog.cs` this is likely an unavoidable consequence of porting a fixed 66-book reference table from the upstream Python library, so splitting it may not be worth the diff noise — but if it's ever hand-edited rather than regenerated, it's worth confirming it isn't hiding logic that belongs elsewhere.

4. Testing & CI

Project	Source files	Test files
Platform.API	34	16 (Platform.API.Tests)
Platform.API.Models	18	— (covered indirectly)
Platform.SDK.Services	12	6 (Platform.SDK.Services.Tests)
Platform.SDK.Components	8	8 (Platform.SDK.Components.Tests)
YouVersion.UsfmReferences	6	5 (YouVersion.UsfmReferences.Tests)
PlatformTestApp	5	6 (PlatformTestApp.Tests) — **not run in CI**

STRENGTH — CI TESTS THE REAL NUGET CONSUMER EXPERIENCE, NOT JUST PROJECT REFERENCES

`.github/workflows/ci.yml` builds every library in dependency order, `dotnet pack`s them to a local feed, and then builds `PlatformTestApp` against the *packed packages* via a local `nuget.config` source rather than project references. That means CI actually catches the class of bug where something works via `ProjectReference` but breaks for a real package consumer (missing `InternalsVisibleTo`, wrong `PackagePath`, a type left `internal` that should be public). This is a meaningfully more rigorous CI setup than most SDK repos bother with.

GAP — PLATFORMTESTAPP.TESTS EXISTS BUT IS NEVER EXECUTED IN CI

MEDIUM

`PlatformTestApp.Tests` contains real, non-trivial tests (`CircuitSessionKeyAccessorTests`, `SessionTokenProviderTests` — exactly the per-user token isolation logic the changelog flags as having fixed a cross-user OAuth token leakage bug). But `ci.yml`'s `Test` step only runs the four library test projects; it never calls `dotnet test PlatformTestApp.Tests`. This is the single highest-value, lowest-effort fix in this report: a one-line addition to the workflow closes a real coverage gap on the exact code (session-scoped token storage) that previously caused a security-relevant bug.

IMPROVEMENT — NO CODE COVERAGE REPORTING OR THRESHOLD GATE

LOW

`coverlet.collector` is referenced in every test project, so coverage data is being generated on every `dotnet test` run, but nothing in CI collects, reports, or gates on it. Wiring `coverlet` output

> into a CI summary (or a badge) would make the existing investment in tests visible and catch coverage regressions on new code.

5. CI/CD, Release Process & Supply Chain

STRENGTH — VERSION MANAGEMENT HAS ZERO MANUAL BOOKKEEPING

MinVer derives every package's version from git tags (`vX.Y.Z`), so there's no hand-edited version string to drift out of sync across five `.csproj` files. Untagged builds get an automatic prerelease suffix, so local/CI builds are always valid without accidentally being mistaken for a real release.

DOCUMENTATION IS STALE RELATIVE TO ACTUAL RELEASE STATE

LOW

`README.md` 's "Versioning & Releases" section states: *"Status: no tagged release has been cut yet... every package currently builds as a 0.0.0-alpha prerelease."* That's no longer true — the repository has three tags (`v0.1.0` , `v0.1.1` , `v0.1.2`), and `CHANGELOG.md` itself documents a shipped `[0.1.0]` release. This is a minor but visible inconsistency for anyone reading the README before deciding whether the packages are usable yet.

GAP — NO DEPENDENCY OR SECURITY SCANNING

MEDIUM

There's no `dependabot.yml` , CodeQL workflow, or `dotnet list package --vulnerable` check anywhere in `.github/` . For a package that ships OAuth token handling and pulls in `Microsoft.Extensions.Caching.StackExchangeRedis` and several other transitive dependencies, this is the most consequential gap in the CI setup — it's the difference between finding out about a CVE in a dependency from a security advisory versus from an automated PR.

IMPROVEMENT — RELEASE AUTOMATION IS DUPLICATED ACROSS FOUR SCRIPTS

LOW

`Publish-NuGetRelease.ps1` (265 lines) / `publish-nuget-release.sh` (228 lines) and `Rebuild-Packages.ps1` (98 lines) / `rebuild-nuget-packages.sh` (127 lines) implement the same two operations twice — once in PowerShell, once in Bash — for cross-platform convenience. That's a reasonable trade-off for a solo maintainer working across OSes, but it's ~720 lines of release logic that has to be kept in sync by hand with no test coverage of its own; a divergence between the two would likely only surface the next time someone happens to use the less-recently-run one. Since `nuget-publish.yml` already automates the real publish path via GitHub Releases, it may be worth clarifying in the scripts' own header comments whether they're meant as the source of truth or as a local dry-run/manual fallback — right now that intent isn't stated anywhere.

6. Documentation

STRENGTH — DOCUMENTATION IS UNUSUALLY HONEST ABOUT TRADEOFFS AND GOTCHAS

`CHANGELOG.md` reads like a postmortem log rather than a marketing summary — entries explain the *compounding causes* of bugs (e.g. the Data Exchange approval page never appearing was two separate bugs stacked together), note which code paths are "confirmed reachable via live sign-in — don't remove without re-verifying," and call out breaking changes to unreleased APIs explicitly. `docs/oauth-guide.md` (~21 KB) is a full walkthrough of a notoriously fiddly integration (OAuth/PKCE plus a multi-shaped consent callback), which is exactly the kind of documentation that's normally missing until a maintainer gets tired of answering the same support question.

STRENGTH — PER-PACKAGE READMES ARE WRITTEN FOR THE NUGET.ORG READER, NOT JUST GITHUB

The changelog shows deliberate attention to how documentation renders *off* GitHub — converting relative doc links to absolute GitHub URLs specifically because NuGet.org's README renderer doesn't resolve repo-relative paths. Most projects only notice this after a user complains about a dead link on the package page.

GAP — NO CONTRIBUTING.MD, ISSUE TEMPLATES, OR SECURITY.MD

LOW

The project is clearly ready for outside contributors (MIT license, CI badge, clean history), but there's nothing telling a first-time contributor how to propose a change, what the PR expectations are (e.g. "update CHANGELOG.md," "run `dotnet test` locally first"), or where to privately report a security issue (relevant here specifically because this SDK handles OAuth tokens). This is low-effort to add and directly lowers the barrier for the project's first external contributor.

7. Priority Recommendations

#	Recommendation	Effort	Impact
1	Add `dotnet test PlatformTestApp.Tests` to `ci.yml`	Trivial	High — closes a real gap on previously-buggy token-isolation code
2	Add Dependabot (or equivalent) for NuGet + GitHub Actions dependencies	Low	High — this SDK handles OAuth tokens; supply-chain visibility matters
3	Fix the README "no tagged release" claim to match the three existing tags	Trivial	Medium — first-impression accuracy for new adopters
4	Add a coverage report/badge from the coverlet data already being collected	Low	Medium
5	Extract `PlatformTestApp/Program.cs`'s callback-dispatch middleware into named, testable handlers	Medium	Medium — prevents this from becoming unmaintainable as more callback shapes are added
6	Add CONTRIBUTING.md and SECURITY.md	Low	Low-Medium — lowers barrier to first external contribution
7	State the intended relationship between the release scripts and `nuget-publish.yml` (source of truth vs. local fallback)	Trivial	Low

Overall assessment: this reads as a repository built by someone who has been burned by OAuth token races, cross-user session leaks, and dead documentation links before, and has since built process (architecture-fitness tests, a real changelog, deterministic builds) to stop those specific mistakes from recurring. The gaps that remain are the ones that only become visible once a project has outside contributors or a security incident — which is a reasonable place for a pre-1.0, single-maintainer SDK to be, and a solid foundation to build that next stage on.

Appendix A: How the Test Suite Enforces Application Architecture

The layered dependency diagram in Section 2 (`Models → API → Services → Components → PlatformTestApp`) is drawn as a convention in the README, but only one piece of it is actually mechanically enforced: the rule that UI-facing projects must reach the Platform API only through `Platform.SDK.Services`, never by calling `Platform.API.Clients` directly. This appendix details the single test responsible for that enforcement, what it does and does not catch, and why it's necessary at all given that project references already exist.

A.1 Why the compiler doesn't already enforce this

`IBibleClient`, `IPassageClient`, and `IHighlightClient` (defined in `Platform.API.Clients`) are **public** interfaces, not `internal`. They have to be public, because `Platform.API`'s own DI extension (`AddYouVersionApiClients`) registers them for consumers who intentionally skip the Services/Components layers and want to call the typed HTTP clients directly (this is a documented, supported usage pattern per the README's "which package(s) do I need" table).

That public surface has a side effect: nothing in the project-reference graph stops `Platform.SDK.Components` or `PlatformTestApp` from injecting `IBibleClient` directly. Both projects already have a transitive reference path to `Platform.API` (Components → Services → API), so the compiler will happily resolve the type. The layering rule is therefore a convention that is only as strong as the discipline of whoever is writing new code — which is exactly the gap this test is designed to close.

A.2 The test: `ApiClientBoundaryTests.cs`

Location: `Platform.API.Tests/Architecture/ApiClientBoundaryTests.cs`. It is the **only** architecture-fitness test in the solution — no equivalent test exists for any other layer boundary (e.g. nothing currently checks that `Platform.API.Models` stays free of dependencies, or that `Platform.SDK.Services` never reaches forward into `PlatformTestApp`).

```
[Fact]
public void UiProjects_ShouldNotDirectlyReference_PlatformApiClientInterfaces()
{
    var repoRoot = FindRepositoryRoot();
    var targetDirectories = new[]
    {
        Path.Combine(repoRoot, "Platform.SDK.Components"),
        Path.Combine(repoRoot, "PlatformTestApp")
    };
```

```
    var forbiddenPatterns = new[]
    {
        @"\busing\s+Platform\.API\.Clients\s*;",
        @"@inject\s+I(?:Bible|Passage|Highlight)Client\b",
        @"\bI(?:Bible|Passage|Highlight)Client\b"
    };

    var violations = new List<string>();

    foreach (var dir in targetDirectories)
    {
        if (!Directory.Exists(dir))
            continue;

        var files = Directory.EnumerateFiles(dir, "*.*", SearchOption.AllDirectories)
            .Where(f => !f.Contains($"
{Path.DirectorySeparatorChar}bin{Path.DirectorySeparatorChar}",
StringComparison.Ordinal)
                     && !f.Contains($"
{Path.DirectorySeparatorChar}obj{Path.DirectorySeparatorChar}",
StringComparison.Ordinal))
            .Where(f => f.EndsWith(".cs", StringComparison.OrdinalIgnoreCase)
                     || f.EndsWith(".razor", StringComparison.OrdinalIgnoreCase));

        foreach (var file in files)
        {
            var text = File.ReadAllText(file);
            if (forbiddenPatterns.Any(pattern => Regex.IsMatch(text, pattern)))
                violations.Add(Path.GetRelativePath(repoRoot, file));
        }
    }

    violations.Should().BeEmpty(
        "UI and app projects should call YouVersion only through
Platform.SDK.Services abstractions.");
}
```

A.3 Mechanism

1. **Scope.** Walks exactly two directories: `Platform.SDK.Components` and `PlatformTestApp` — the two projects sitting above `Platform.SDK.Services` in the intended call chain.
2. **File selection.** Enumerates every file recursively, excludes anything under a `bin` or `obj` path, keeps only `.cs` and `.razor` files (so both code-behind and markup are covered).
3. **Detection.** Reads each file's raw text and regex-matches against three patterns, each targeting a different way the boundary could be crossed:

- `\busing\s+Platform\.API\.Clients\s*;` — a C# `using` import of the whole forbidden namespace.
- `@inject\s+I(?:Bible|Passage|Highlight)Client\b` — the Razor-specific `@inject` directive, since Blazor components often pull dependencies this way rather than through a constructor.
- `\bI(?:Bible|Passage|Highlight)Client\b` — a catch-all for the type name appearing anywhere else (constructor parameter, field declaration, cast, fully-qualified reference without a `using`).

4. **Failure mode.** Any match in any file adds that file's relative path to a `violations` list. A single `FluentAssertions` assertion (`violations.Should().BeEmpty()`) fails the entire test, and therefore the build, if the list is non-empty.

A.4 What it catches vs. what it misses

Catches	Misses
• A component or page directly injecting `IBibleClient` / `IPassageClient` / `IHighlightClient` instead of going through a `Platform.SDK.Services` service. • Both C#-side and Razor-side (`@inject`) violations. • A bare `using Platform.API.Clients;` import even before any type from it is used.	• New client interfaces added later (e.g. a future `IVersionClient`) are invisible to this test until someone remembers to add it to `forbiddenPatterns` by hand. • False positives are possible: the pattern would flag the interface name appearing inside a comment or a string literal, since matching is plain text, not AST/symbol-aware. • False negatives are possible for any reference the regex doesn't anticipate (e.g. an unusual whitespace/formatting split across lines that defeats `\b...\b` matching). • Only checks the two named directories — it does not verify any other boundary in the stack, such as `Platform.API.Models` staying dependency-free or `Platform.SDK.Services` not reaching into app-layer code.

A.5 Why it matters despite the limitations

Text-pattern matching is a low-effort, zero-dependency way to enforce an architectural rule that would otherwise rely entirely on code review discipline. Given that the interfaces it guards are necessarily public (Section A.1), this test is the only thing standing between "documented convention" and "compiler-verifiable rule" for the one layering boundary the project has decided matters most: keeping the UI layer's dependency surface limited to `Platform.SDK.Services`. It runs as an ordinary xUnit fact

inside `Platform.API.Tests`, which `ci.yml` does execute on every push, so a violation fails CI rather than silently merging.